

GEXLens — Manuál pro správce a vývojáře

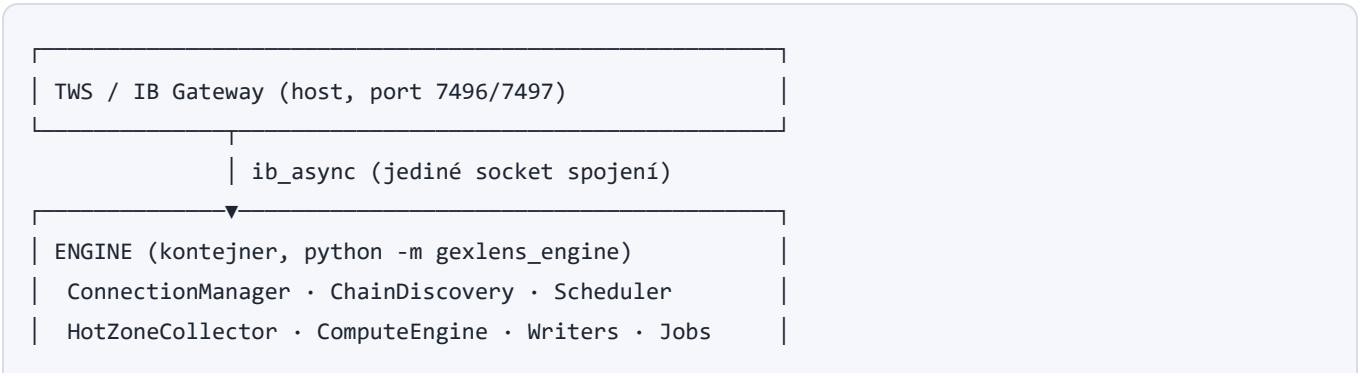
Verze 1.0 · červenec 2026 · interní dokumentace — není dostupná v aplikaci

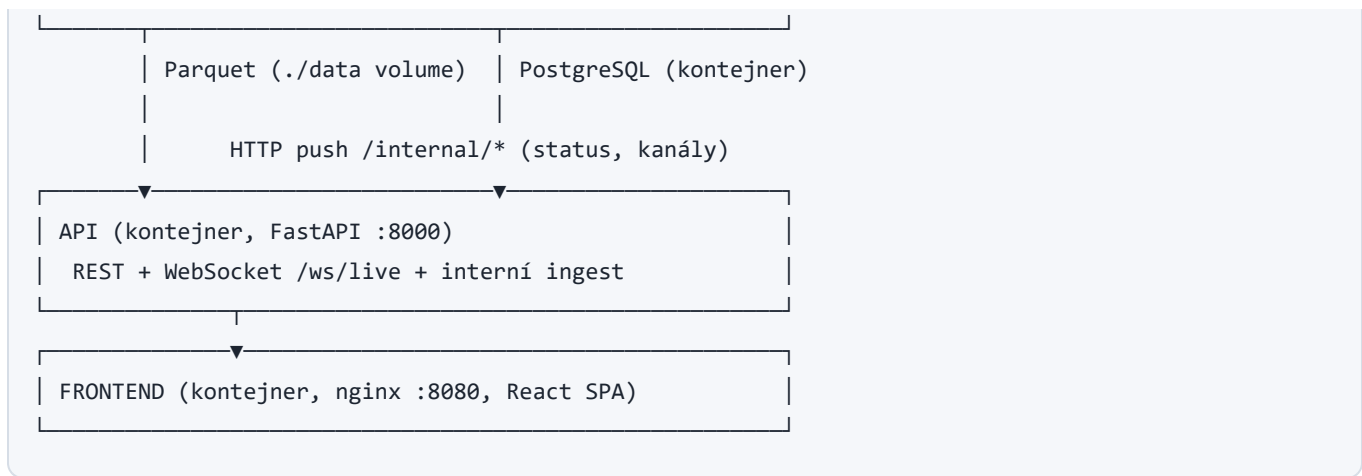
Technický popis architektury, provozu, konfigurace a vývoje aplikace GEXLens. Uživatelská příručka: `UZIVATELSKY-MANUAL.md`. Zdroj pravdy funkčních požadavků: [docs/SPEC.md](#) (v2.0); architektonická rozhodnutí v [docs/adr/](#).

Obsah

- 1. [Architektura](#)
- 2. [Struktura repozitáře](#)
- 3. [Provoz \(docker compose\)](#)
- 4. [Konfigurace — kompletní reference](#)
- 5. [Engine — datová pipeline](#)
- 6. [Datové formáty a persistence](#)
- 7. [API reference](#)
- 8. [Frontend](#)
- 9. [Vývojové prostředí](#)
- 10. [Testy a CI](#)
- 11. [Známé limity účtu a otevřené body](#)
- 12. [Diagnostika a údržba](#)

1. Architektura





Klíčové vlastnosti:

- **Engine a API jsou oddělené procesy.** Engine počítá a zapisuje; API jen čte storage a přeposílá live push z enginu (interní HTTP ingest → StatusStore + LiveHub → WebSocket klientům).
- **Vše lokální** — API CORS povoluje jen `localhost / 127.0.0.1`; žádná telemetrie.
- Engine se z kontejneru připojuje na TWS na hostiteli přes `host.docker.internal`.

2. Struktura repozitáře

```

GEX/
├ engine/                                Python 3.12 balík gexlens_engine
│   └─ src/gexlens_engine/
│       ├── config.py                    Pydantic Settings (.env, GEXLENS_*)
│       ├── ibkr/                        connection, discovery, scheduler, hotzone,
│       │   │                             underlying (bary+pacing), mock (pro testy)
│       ├── compute/                     gex, levels, heatmap, walls, cumdelta, profile
│       ├── storage/                     parquet_store, oi_archive, retention, meta
│       ├── adapters.py                  produkční ib_async adaptéry + HttpPublisher
│       ├── runtime.py                   EngineRuntime – minutový cyklus (testovatelný)
│       └─ __main__.py                   vstupní bod: discovery→archiv→smyčka
├ api/                                  Python balík gexlens_api (FastAPI)
│   └─ src/gexlens_api/                  main (routy+WS), data, heatmap (vektorizace),
│       │                               live (hub), status, crud, alerts, meta_repo
├ frontend/                             React + TypeScript + Vite
│   └─ src/                             components/, heatmap/, replay/, panels/,
│       │                               profile/, annotations/, state/, api/
├ docs/                                 SPEC.md, adr/, manual/
├ docker/                               Dockerfiles + nginx.conf
└─ compose.yml                           celý stack
  
```

└ scripts/
└ Makefile

bootstrap, start skript pro plochu
test / run / run-api / run-frontend / run-engine

Pravidla vývoje jsou v [CLAUDE.md](#) : práce po GitHub issues, golden testy výpočtů, IBKR se v CI nikdy nevolá živě (mock vrstva `engine/ibkr/mock.py`), komentáře česky / identifikátory anglicky.

3. Provoz (docker compose)

docker compose up -d --build
docker compose ps
docker compose logs -f engine
docker compose stop
docker compose down

start / rebuild
stav služeb
živé logy enginu
zastavení (data zůstávají)
odstranění kontejnerů (volume pgdata zůstává)

Služba	Port (host)	Poznámka
frontend	8080	nginx, SPA + <code>/manual/</code> wiki
api	8000	FastAPI, OpenAPI na <code>/docs</code>
postgres	55432	⚠ záměrně ne 5432/5433 — na vývojovém PC běží nativní PostgreSQL na obou
engine	—	bez portu; TWS přes <code>host.docker.internal:7496</code>

Data: Parquet v `./data` (bind mount, sdílené engine↔API), PostgreSQL ve volume `pgdata` .
Zálohovat stačí `./data` + `pg_dump` (hlavně tabulku `oi_eod` , která se nikdy nemaže).

4. Konfigurace — kompletní reference

Zdroj: proměnné prostředí `GEXLENS_*` a `.env` (viz `.env.example`). Validuje se při startu — nevalidní hodnota = engine odmítne nastartovat se srozumitelnou chybou.

Proměnná	Default	Význam
<code>GEXLENS_IBKR_HOST</code>	127.0.0.1	V compose přepsáno na <code>host.docker.internal</code>

Proměnná	Default	Význam
GEXLENS_IBKR_PORT	7496	7496 live / 7497 paper (TWS); 4001/4002 (Gateway)
GEXLENS_IBKR_CLIENT_ID	1	
GEXLENS_MARKET_DATA_TYPE	1	1=live; delayed engine odmítá
GEXLENS_CONNECT_TIMEOUT_S	10	
GEXLENS_RECONNECT_BACKOFF_BASE_S / _MAX_S	2 / 60	Exponenciální reconnect
GEXLENS_STRIKE_RANGE_POINTS	200	Výchozí denní obálka spot ± X (ADR-0002)
GEXLENS_STRIKE_RANGE_EXPAND_THRESHOLD	0.25	Rozšíření při přiblížení k okraji
GEXLENS_STRIKE_RANGE_MAX_POINTS	800	Strop šířky obálky ($\geq 2 \times$ base)
GEXLENS_BATCH_SIZE	80	Dávka rotačních subskripcí
GEXLENS_BATCH_TIMEOUT_S	4	Čekání na kompletní data kontraktu
GEXLENS_WINGS_SWEEP_EVERY	3	Křídla každý k-tý cyklus
GEXLENS_ATM_SWEEP_WIDTH	30	ATM ± N strikes každý cyklus
GEXLENS_REPAIR_MAX_ATTEMPTS	3	Retry repair fronty za sweep
GEXLENS_MARKET_DATA_LINES	100	Kapacita lines (účet ≥ 150 , ADR-0001)
GEXLENS_HOT_ZONE_WIDTH	15	Cílová šířka hot zóny (degraduje dle účtu)
GEXLENS_TICK_BY_TICK_MAX_STREAMS	5	Naměřený limit účtu (ADR-0001)
GEXLENS_DATABASE_URL	postgres localhost	V compose směřuje na službu postgres

Proměnná	Default	Význam
GEXLENS_DATA_DIR	data	Kořen Parquet partic
GEXLENS_RETENTION_DAYS	14	Purge okno (oi_eod se nikdy nemaže)
GEXLENS_DISK_LIMIT_GB	2	Alert při překročení
GEXLENS_RETENTION_PURGE_TIME_UTC	21:30	Čas nočního purge
GEXLENS_API_BASE	http://127.0.0.1:8000	Kam engine pushuje (v compose <code>http://api:8000</code>)

Frontend build-time: `VITE_API_BASE` (nginx build arg, default `http://127.0.0.1:8000`).

5. Engine — datová pipeline

Minutový cyklus (`runtime.EngineRuntime.run_cycle`):

1. **Sweep** — `SubscriptionScheduler` projede řetězec v dávkách (ATM±30 každý cyklus, křídla každý 3.), nekompletní kontrakty přes repair frontu, výsledek do in-memory cache.
2. **Snapshot** — cache → `SnapshotRow` (OI z ranního archivu) → atomický zápis Parquet.
3. **Výpočty** — GEX per strike (naivní dealer model, vyměnitelná strategie) → levels (flip interpolovaně, walls, centroid) → zápis do `derived/levels`.
4. **Cum Δ** — bar větev ($\Delta \text{Vol} \times \text{midpoint test} \times \Delta \times M$); hot zóna tick-by-tick přispívá průběžně. `close_minute` → `derived/flow`.
5. **Bary podkladu** — 5s `reqRealTimeBars` agregované na 1min → `derived/bars`.
6. **Push do API** — `/internal/status` + kanály `levels.*`, `flow.*`, `price.*`.

Další joby: **OI archiv** při startu + retry à 30 min dokud den nemá data (alert `oi_missing` při selhání); **auto-rozšíření obálky strikes** (grow-only, capped → alert); **noční retention purge** po `RETENTION_PURGE_TIME_UTC`.

Odolnost: ConnectionManager watchdog (heartbeat + exponenciální reconnect + plná resubskripce), discovery s timeoutem a retry (sec-def farm výpadky), výjimka v cyklu nikdy neshodí smyčku, pacing guard historical requestů ($\leq 60/10$ min, dedup, priorita).

6. Datové formáty a persistence

Parquet (`GEXLENS_DATA_DIR` , retence 14 dní)

Partice	Schéma
<code>snapshots/{sym}/{expiry}/{YYYY-MM-DD}.parquet</code>	ts_min, strike, right, bid, ask, last, volume, iv, delta, gamma, theta, vega, oi, stale_age
<code>ticks/{sym}/{YYYY-MM-DD}.parquet</code>	ts, conld, price, size, side
<code>derived/{sym}/{expiry}/levels/{date}.parquet</code>	ts_min, flip, call_wall, put_wall, centroid, total_gex
<code>derived/{sym}/flow/{date}.parquet</code>	ts_min, flow_delta, cum_delta
<code>derived/{sym}/bars/{date}.parquet</code>	ts_min, open, high, low, close, volume

Zápis je **atomický** (temp + rename) — po pádu procesu nikdy nezůstane částečný soubor; osiřelé `.tmp` se uklízí při dalším zápisu. Writer po restartu navazuje na rozepsaný den.

PostgreSQL

Tabulka	Účel
<code>oi_eod(symbol, expiry, strike, right, date, oi)</code>	Věčný OI archiv — žádná retence, žádné delete API
<code>watchlist</code> , <code>alerts</code> , <code>annotations</code> , <code>settings</code>	CRUD přes API

7. API reference

Interaktivní dokumentace: `http://127.0.0.1:8000/docs` (OpenAPI).

REST

Endpoint	Popis
<code>GET /health</code> , <code>GET /status</code>	Liveness; agregovaný stav pipeline

Endpoint	Popis
GET /instruments , GET /instruments/{sym}/expiries	Dostupné symboly/expirace (ze storage)
GET /snapshots/{sym}/{expiry}?date&mode&scale&norm&from&to&raw	Heatmap matice jako Arrow IPC stream ; raw=true = surová partice
GET /levels/{sym}/{expiry}?date	Časová řada flip/walls/centroid
GET /profile/{sym}/{expiry}?date&ts&variant&oi_weight&spot	Strike profil k okamžiku
GET /flow/{sym}?date	CumΔ + OptVol + Vol řady
GET /replay/{sym}/{expiry}/{date}	Kompletní denní balík (levels/flow/bars JSON + snapshoty base64 Arrow)
CRUD /watchlist , /alerts , /annotations?symbol&date , /settings	PostgreSQL persistence
POST /internal/status , POST /internal/publish	Ingest z enginu (nechráněné — API bindí jen na localhost)

WebSocket /ws/live

Protokol: klient pošle {"action": "subscribe", "channels": ["status", "price.ES", "levels.*"]} (podpora trailing wildcard), server vrací ack a pushuje {"channel": ..., "data": ...} . Backpressure: fronta 100 zpráv per klient, při zaplnění se zahazují nejstarší framy. Kanály: status , price.{sym} , snapshot.{sym}.{expiry} , levels.* , flow.* , alerts , news .

8. Frontend

- **Heatmapa:** data → offscreen bitmapa (překreslení jen při změně dat/módu), pan/zoom = GPU drawImage → 60 fps nezávisle na objemu; overlay canvas kreslí vektory (cena/svíčky, levels, walls, sessions, crosshair, anotace).
- **Replay:** /replay se stáhne jednou, apache-arrow dekáduje snapshoty, celý den se předpočítá v paměti (vč. profilu per minuta) — přetáčení je čisté krájení typed arrays. Timestampy se normalizují (canonicalTs — Arrow epoch vs. JSON ISO).
- **Stav:** React kontexty AppState (status z WS + REST initial fetch, view, téma, alerty) a Crosshair (sdílený všemi panely).

- **OI fallback:** při nulovém OI staví heatmapu z volume (engine mezitím posílá alert `oi_missing`).
- Wiki/manuál: statické HTML v `frontend/public/manual/` (generované z MD, viz níže) — servíruje ho vite dev i nginx.

9. Vývojové prostředí

Prerekvizity: [uv](#) (stáhne Python 3.12 sám), Node.js ≥ 20 , Docker (pro PG integrační test lokálně volitelně).

```
uv sync --all-packages                # Python workspace (engine + api)
uv run ruff check .; uv run ruff format . # lint/format
uv run mypy engine/src engine/tests api/src api/tests
uv run pytest                          # PG integrační test se přeskočí bez GEXLENS_TEST_PG_DSN

cd frontend; npm ci; npm run lint; npm test; npm run build
```

Dev servery: `make run-api` (uvicorn :8000), `make run-frontend` (vite :5173), `make run-engine` (vyžaduje TWS). CORS povoluje i :5173.

Regenerace manuálů (MD → HTML pro in-app wiki → PDF): `powershell scripts/build-manual.ps1` (vyžaduje Edge; PDF vzniká headless tiskem).

Konvence: feature branch `feat/{issue}-slug` / `fix/...`, PR s `closes #N`, merge po zeleném CI. Výpočty vždy s golden testy v `engine/tests/golden/` (ručně spočtené hodnoty, výpočet dokumentovaný v `description`).

10. Testy a CI

- **Python** (~160): jednotkové + golden (GEX, levels, heatmap módy, walls, Cum Δ , profil), mock-based integrační (scheduler, hot zóna, runtime), PG integrační (v CI přes service kontejner), **e2e smoke** — deterministický referenční den přes celou pipeline engine→storage→API proti golden hodnotám.
- **Frontend** (~58): jednotkové (geometrie, barvy, contours, slice), komponentové (jsdom + testing-library, PointerEvent polyfill), Arrow round-trip loaderu, **e2e render smoke** (App nad /replay balíkem), vizuální regresní snapshoty renderu.

- **CI** (GitHub Actions, na každý PR): python job (ruff, mypy strict, pytest + PostgreSQL service), frontend job (eslint, prettier, vitest, build). Výkonnostní testy s tvrdým limitem běží jen lokálně (`ci` env skip).

11. Známé limity účtu a otevřené body

Z [ADR-0001](#) (měřeno živě na účtu):

Limit	Hodnota	Dopad
Tick-by-tick streamy	5	Hot zóna degraduje z $ATM \pm 15$ na $\sim ATM \pm 1$; zbytek klasifikuje midpoint test. Rozšíření = IBKR Quote Booster.
Market data lines	≥ 150	Dávka 80 má rezervu
FOP OI (tick 588)	intraday nechodí vůbec	GEX bez OI do ranního CME okna; mitigace: retry à 30 min + alert + volume fallback heatmapy. Otevřené: issue #65 — ověřit ranní okno; případně rozhodnout alternativu.

[ADR-0002](#): obálka strikes je grow-only (křídla se neztrácejí), strop šířky s alertem.

12. Diagnostika a údržba

Situace	Postup
Engine offline	<code>docker compose logs engine</code> — hledej stav ConnectionManageru; ověř TWS (API zapnuté, port, Trusted IP). Warning 2110/2103 = výpadek TWS↔IB, vyřeší se sám.
Prázdné GEX/walls	Zkontroluj <code>oi_eod</code> pro dnešek: <code>docker compose exec postgres psql -U gexlens -c "select date, count(*) from oi_eod group by 1 order by 1 desc limit 5"</code> — pokud dnešek chybí, viz issue #65.
Vysoké <code>Repair</code> / <code>Stale</code>	Konkrétní kontrakty bez dat — často nelikvidní křídla; zvyš <code>BATCH_TIMEOUT_S</code> nebo zmenši obálku.
Disk roste	Retention běží nočně; ručně: smaž staré partice v <code>./data</code> (nikdy <code>oi_eod</code>).

Situace	Postup
Reset prostředí	<code>docker compose down</code> , smaž <code>./data</code> (přijdeš o 14denní okno, ne o Ol archiv ve volume <code>pgdata</code>), <code>docker compose up -d --build</code> .
Málo dat po restartu	Writer navazuje na rozepsaný den — mezera zůstane jen za dobu výpadku.
Změna portu TWS	Settings v aplikaci, nebo <code>.env</code> + <code>docker compose up -d engine</code> .

Interní dokument. Uživatelská příručka: `UZIVATELSKY-MANUAL.md` (dostupná i v aplikaci jako Wiki).